

Parking Allotment Backend — Project Guide

Simple & Comprehensive Overview for Technical Interview & System Defense

1. System Overview

A REST API built with **FastAPI** and **PostgreSQL** for a multi-floor parking facility. It handles vehicle registration, smart slot allocation by preference, slot release with duration calculation, and real-time occupancy tracking.

The 3 Core Concepts:

- **Slot:** A physical parking space (e.g. A-1, B-10), typed as BIKE, CAR, or EV, either AVAILABLE or OCCUPIED.
- **Vehicle:** A registered vehicle with an uppercase license plate, owner name, and type.
- **Allocation:** The record linking a vehicle to a slot with entry/exit timestamps and status (ACTIVE / RELEASED).

2. The 7 API Endpoints

- **POST /slots (201):** Creates a parking slot with format <FLOOR>-<NUMBER> (e.g. A-1, B-10). Returns 409 if duplicate.
- **GET /slots (200):** Lists all slots in numeric order. Optional filters by status and type.
- **POST /vehicles (201):** Registers a vehicle plate. Returns 409 if already registered.
- **GET /vehicles (200):** Lists all registered vehicles with current slot and is_parked status (Single query, no N+1).
- **POST /allocate (201):** Automatically allots the lowest available compatible slot.
- **POST /release (200):** Frees the slot and computes parking duration in minutes (rounded up).
- **GET /allocations (200):** Lists active parking sessions ordered by newest first.

3. Essential Business Rules

A. Slot Ordering (§4.1):

Lowest floor first, then lowest numerical slot (A-1 < A-2 < A-9 < A-10 < A-100 < B-1). Storing slot number as plain text causes alphabetical bugs ('A-10' before 'A-2'). We solved this by storing floor and slot_num (integer) in separate indexed database columns.

B. Vehicle Preferences (§4.2):

- **BIKE:** Only parks in BIKE slots.
- **CAR:** Only parks in CAR slots (never in EV slots; chargers are reserved).
- **EV:** Prefers EV slots; if full, falls back to CAR slots (does not charge).

C. Concurrency Control (§4.4):

To prevent two concurrent requests from taking the same slot at the same millisecond, we use **Row-Level Locking** (SELECT ... FOR UPDATE) combined with **Database Partial Unique Indexes** (WHERE status = 'ACTIVE').

D. Error Envelope (§5.8):

All non-2xx responses use the identical JSON structure: { "error": { "code": "...", "message": "...", "details": null } }.

4. Technical Interview Defense ('What Breaks If We Delete This Line?')

- **query = query.with_for_update()**: Without row locking, parallel requests read the same free slot simultaneously, leading to double booking race conditions.
- **CREATE UNIQUE INDEX ... WHERE status = 'ACTIVE'**: Enforces database-level authority so no vehicle or slot can ever hold two active parkings at the same time.
- **v = v.strip().upper()**: Guarantees plate normalization so 'mh12ab1234' and 'MH12AB1234' are treated as the exact same vehicle.
- **duration_minutes = max(1, math.ceil(delta / 60))**: Ensures parking duration is rounded up to whole minutes as required by the specification.

5. Quick Commands

- **Start App**: `python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload`
- **Run Tests**: `python -m pytest -v` (22/22 tests passing)
- **Swagger Docs**: `http://localhost:8000/docs`